

Chapter 4: Understanding Loss Functions and Training Dynamics

Objective: To understand the role of loss functions in training a neural network, how to interpret loss values during training, and the process of optimization. This session will also involve analyzing and tweaking the training process to improve model performance.

1. What Are Loss Functions?

A loss function measures how well the model's predictions match the true labels. During training, the optimizer updates the model's weights to minimize this loss.

Tasks:

1. Research the difference between **Cross-Entropy Loss** and **Mean Squared Error (MSE)**. Why is Cross-Entropy Loss more suitable for classification tasks like CIFAR-10?
2. Look at the formula for Cross-Entropy Loss. What does it penalize? How does it handle incorrect predictions with high confidence?

Questions to Explore:

- What does a lower loss value mean for the model's performance?
- How is the loss function connected to the gradients used in backpropagation?

Answer:

1. Cross-Entropy vs. MSE:

- **MSE:** Measures squared differences; best for continuous regression tasks.
- **Cross-Entropy:** Measures differences between probability distributions; best for classification (like CIFAR-10) as it focuses on the predicted probability of the true class.

2. Penalization & Confident Errors:

- It penalizes assigning low probabilities to the correct class.
- For "confidently wrong" predictions (assigning near 0 probability to the true class), the penalty $-\log(\approx 0)$ approaches infinity.

Questions to Explore:

- **Lower Loss Value:** The model predicts the true labels more accurately and with higher certainty.

- **Backpropagation:** The loss is the starting point. We calculate its gradients relative to each weight, telling the optimizer how to adjust weights to reduce future errors.

2. Training Dynamics

Tasks:

1. Look at the training and validation loss/accuracy plots from Chapter 3.
Answer the following:
 - Are the training and validation loss decreasing over epochs?
 - Is the gap between training and validation accuracy increasing? What might this indicate?
2. Research the terms **overfitting** and **underfitting**. Based on your plots, is your model showing signs of either? Why?

Questions to Explore:

- What might cause the validation accuracy to plateau while training accuracy continues to improve?
- How could you adjust the model or training process to address overfitting or underfitting?

Answer:

Tasks:

- 1. training loss is continually decreasing (as training accuracy keeps rising), but validation loss likely stopped decreasing after epoch 10 (as validation accuracy plateaus).
- 2. Yes, a widening gap indicates that the model is overfitting to the training data.

Questions to Explore:

- 1. The model is memorizing training noise and specific details, losing its ability to generalize.
- 2. Use Early Stopping, Dropout, or data augmentation to mitigate overfitting; increase model complexity to fix underfitting.

3. Experimenting with Hyperparameters

Hyperparameters like learning rate, batch size, and number of epochs significantly influence the training process.

Tasks:

1. Experiment with different learning rates (e.g., 0.1, 0.01, 0.001). How does a larger or smaller learning rate affect convergence and final accuracy?
2. Vary the batch size (e.g., 32, 64, 128). How does changing the batch size influence training time and performance?
3. Try training the model for fewer or more epochs. At what point does the validation loss start to increase, indicating overfitting?

Programming Exercise: Modify the `model.fit()` call from Chapter 3 to experiment with these hyperparameters. Record your observations for each run.

Question: Parameters vs. Hyperparameters

Definition of a Hyperparameter:

- A hyperparameter is an external configuration variable whose value is set *by the user before the training process begins*. Its optimal value cannot be directly mathematically learned from the data itself.

Difference between Parameters and Hyperparameters:

- **Parameters:** Internal variables that the model learns and updates *automatically* from the data during training (e.g., neuron weights and biases).
- **Hyperparameters:** External settings determined *manually* by the researcher to control how the model learns (e.g., learning rate, number of hidden layers, batch size, epochs).

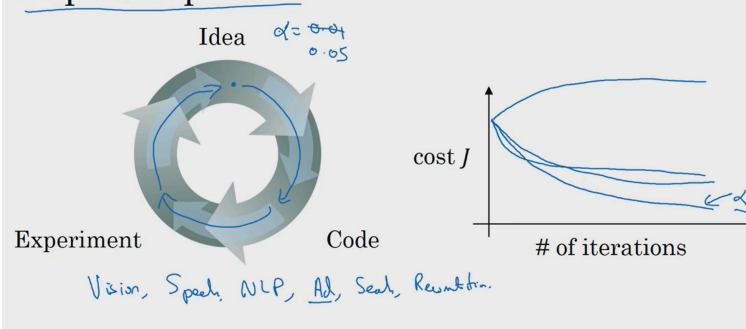
(Reference Images)

What are hyperparameters?

Parameters: $W^{[1]}, b^{[1]}, W^{[2]}, b^{[2]}, W^{[3]}, b^{[3]} \dots$

Hyperparameters: α , τ , L , $n^{[1]}, n^{[2]}, \dots$, choice of activation function, Momentum, mini-batch size, regularizations...

Applied deep learning is a very empirical process



```

In [ ]: # Programming Exercise: Experiment with different hyperparameters

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Flatten, Dense
from tensorflow.keras.optimizers import Adam
from sklearn.model_selection import train_test_split
import matplotlib.pyplot as plt

# 1. Load CIFAR-10 dataset
# Use the dataset prepared in Chapter 2 (including train, validation, and test sets)
(train_images_full, train_labels_full), (test_images,
                                         test_labels) = tf.keras.datasets.cifar10.load_data()

# Normalize pixel values to be between 0 and 1
train_images_full = train_images_full / 255.0
test_images = test_images / 255.0

# Split into train and validation sets (e.g., 80% train, 20% validation)
train_images, val_images, train_labels, val_labels = train_test_split(
    train_images_full, train_labels_full, test_size=0.2, random_state=42
)

def create_and_train_model(learning_rate, batch_size, epochs):
    print(
        f"\n--- Training with LR: {learning_rate}, Batch Size: {batch_size}, Epochs: {epochs}"
    )

    # 1. Redefine the model (ensure each experiment starts from scratch, not continue from previous)
    model = Sequential([
        Flatten(input_shape=(32, 32, 3)),
        Dense(128, activation='relu'),
        Dense(64, activation='relu'),
        Dense(10, activation='softmax')
    ])

    # 2. Compile the model: pass the specific learning_rate here
    model.compile(
        optimizer=Adam(learning_rate=learning_rate),
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )

    # 3. Train the model: pass specific batch_size and epochs here
    history = model.fit(
        train_images,
        train_labels,
        validation_data=(val_images, val_labels),
        epochs=epochs,
        batch_size=batch_size,
        verbose=0 # Set to 0 to disable progress bar printing, only view final results
    )

    # Print final epoch results
    final_train_acc = history.history['accuracy'][-1]
    final_val_acc = history.history['val_accuracy'][-1]
    print(
        f"Final Train Accuracy: {final_train_acc:.4f} | Final Val Accuracy: {final_val_acc:.4f}"
    )

```

```

    return history

# --- Experiment 1: Different Learning Rates (fixed batch_size=64, epochs=15) ---
history_lr_large = create_and_train_model(
    learning_rate=0.1, batch_size=64, epochs=15)
history_lr_good = create_and_train_model(
    learning_rate=0.001, batch_size=64, epochs=15)
history_lr_small = create_and_train_model(
    learning_rate=0.00001, batch_size=64, epochs=15)

# --- Experiment 2: Different Batch Sizes (fixed lr=0.001, epochs=15) ---
history_batch_small = create_and_train_model(
    learning_rate=0.001, batch_size=32, epochs=15)
history_batch_large = create_and_train_model(
    learning_rate=0.001, batch_size=256, epochs=15)

# Visualization: Compare the 5 experiments
plt.figure(figsize=(14, 6))

# 1. Validation Accuracy Comparison
plt.subplot(1, 2, 1)
plt.plot(history_lr_large.history['val_accuracy'], label='LR=0.1 (Batch 64)')
plt.plot(history_lr_good.history['val_accuracy'], label='LR=0.001 (Batch 64)')
plt.plot(history_lr_small.history['val_accuracy'],
         label='LR=0.00001 (Batch 64)')
plt.plot(
    history_batch_small.history['val_accuracy'], label='Batch=32 (LR 0.001)')
plt.plot(
    history_batch_large.history['val_accuracy'], label='Batch=256 (LR 0.001)')
plt.title('Validation Accuracy Comparison')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)

# 2. Validation Loss Comparison
plt.subplot(1, 2, 2)
plt.plot(history_lr_large.history['val_loss'], label='LR=0.1 (Batch 64)')
plt.plot(history_lr_good.history['val_loss'], label='LR=0.001 (Batch 64)')
plt.plot(history_lr_small.history['val_loss'], label='LR=0.00001 (Batch 64)')
plt.plot(history_batch_small.history['val_loss'], label='Batch=32 (LR 0.001)')
plt.plot(history_batch_large.history['val_loss'], label='Batch=256 (LR 0.001)')
plt.title('Validation Loss Comparison')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()

```

```

2026-03-19 07:47:58.622408: E external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:467] Unable to register cuFFT factory: Attempting to register factory for plugin cuFFT when one has already been registered
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
E0000 00:00:1773906478.786618      101 cuda_dnn.cc:8579] Unable to register cuDNN factory: Attempting to register factory for plugin cuDNN when one has already been registered
E0000 00:00:1773906478.841933      101 cuda_blas.cc:1407] Unable to register cuBLAS factory: Attempting to register factory for plugin cuBLAS when one has already been registered
W0000 00:00:1773906479.247040      101 computation_placer.cc:177] computation placer already registered. Please check linkage and avoid linking the same target more than once.
W0000 00:00:1773906479.247078      101 computation_placer.cc:177] computation placer already registered. Please check linkage and avoid linking the same target more than once.
W0000 00:00:1773906479.247080      101 computation_placer.cc:177] computation placer already registered. Please check linkage and avoid linking the same target more than once.
W0000 00:00:1773906479.247083      101 computation_placer.cc:177] computation placer already registered. Please check linkage and avoid linking the same target more than once.

Downloading data from https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz
170498071/170498071 ————— 3s 0us/step

```

```

--- Training with LR: 0.1, Batch Size: 64, Epochs: 15 ---

```

```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/reshaping/flatten.py:37:
UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(**kwargs)
I0000 00:00:1773906508.585384      101 gpu_device.cc:2019] Created device /job:localhost/replica:0/task:0/device:GPU:0 with 15511 MB memory: -> device: 0, name: Tesla P100-PCIE-16GB, pci bus id: 0000:00:04.0, compute capability: 6.0
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1773906512.085156      138 service.cc:152] XLA service 0x7c6f1c008180 initialized for platform CUDA (this does not guarantee that XLA will be used). Devices:
I0000 00:00:1773906512.085191      138 service.cc:160]   StreamExecutor device (0): Tesla P100-PCIE-16GB, Compute Capability 6.0
I0000 00:00:1773906512.318721      138 cuda_dnn.cc:529] Loaded cuDNN version 91002
I0000 00:00:1773906513.031028      138 device_compiler.h:188] Compiled cluster using XLA! This line is logged at most once for the lifetime of the process.
Final Train Accuracy: 0.1006 | Final Val Accuracy: 0.1040

```

```

--- Training with LR: 0.001, Batch Size: 64, Epochs: 15 ---
Final Train Accuracy: 0.5012 | Final Val Accuracy: 0.4697

```

```

--- Training with LR: 1e-05, Batch Size: 64, Epochs: 15 ---
Final Train Accuracy: 0.4146 | Final Val Accuracy: 0.4045

```

```

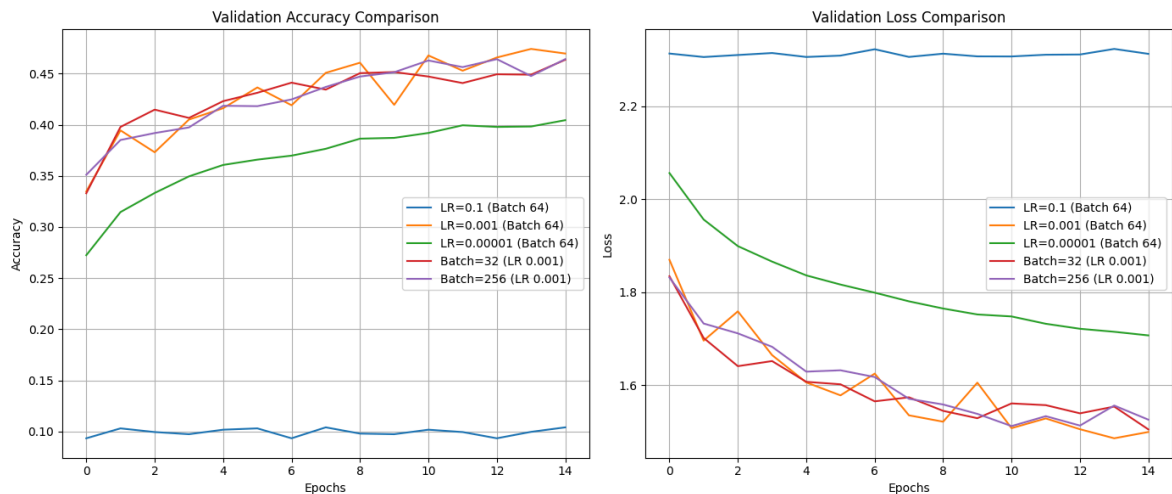
--- Training with LR: 0.001, Batch Size: 32, Epochs: 15 ---
Final Train Accuracy: 0.4910 | Final Val Accuracy: 0.4636

```

```

--- Training with LR: 0.001, Batch Size: 256, Epochs: 15 ---
Final Train Accuracy: 0.4900 | Final Val Accuracy: 0.4642

```



4.1 Regularization Techniques

To improve generalization and prevent overfitting, you can use techniques like dropout and weight regularization.

Tasks:

1. Research **dropout layers**. What do they do during training? How can they help prevent overfitting?
2. Research **L2 regularization** (also called weight decay). What effect does it have on the model's weights?

Programming Exercise:

- Add a dropout layer to your model (e.g., Dropout(0.5) after a hidden layer).
- Add L2 regularization to one of the dense layers (e.g., Dense(units, kernel_regularizer=tf.keras.regularizers.l2(0.01))).
- Retrain the model and compare the training and validation accuracy before and after adding these regularization techniques.

Answer:

1. Dropout layers: During training, dropout randomly deactivates (sets to zero) a fraction of neurons in a layer at each update step. This prevents overfitting by keeping neurons from co-adapting too much, forcing the network to learn more robust, redundant features instead of relying on a few specific nodes.

2. L2 regularization: L2 regularization (weight decay) adds a penalty term proportional to the square of the layer's weights to the loss function. This pushes the model's weights to be smaller and closer to zero, resulting in a simpler, smoother model that is less sensitive to noise in the training set.

In [2]: `# Programming Exercise: Add regularization techniques`

```

from tensorflow.keras.layers import Dropout, Flatten, Dense
from tensorflow.keras.regularizers import l2
from tensorflow.keras.models import Sequential
from tensorflow.keras.optimizers import Adam

print("\n--- Training Model with Dropout and L2 Regularization ---")

# Define the model with L2 regularization and Dropout
model_reg = Sequential([
    Flatten(input_shape=(32, 32, 3)),
    # Apply L2 Regularization
    Dense(128, activation='relu', kernel_regularizer=l2(0.01)),
    # Apply Dropout Layer (50% probability)
    Dropout(0.5),
    Dense(64, activation='relu'),
    Dense(10, activation='softmax')
])

# Compile the model
model_reg.compile(
    optimizer=Adam(learning_rate=0.001),
    loss='sparse_categorical_crossentropy',
    metrics=['accuracy']
)

# Train the model
history_reg = model_reg.fit(
    train_images,
    train_labels,
    validation_data=(val_images, val_labels),
    epochs=15,
    batch_size=64,
    verbose=1 # Set verbose=1 to observe the training progress
)

# Output the final results
final_train_acc_reg = history_reg.history['accuracy'][-1]
final_val_acc_reg = history_reg.history['val_accuracy'][-1]
print(
    f"Regularized Model -> Final Train Accuracy: {final_train_acc_reg:.4f} | Fin

```

--- Training Model with Dropout and L2 Regularization ---


```

Epoch 1/15
625/625 ————— 5s 4ms/step - accuracy: 0.1430 - loss: 2.8771 - val_
accuracy: 0.2219 - val_loss: 2.0841
Epoch 2/15
625/625 ————— 2s 3ms/step - accuracy: 0.1592 - loss: 2.1994 - val_
accuracy: 0.2143 - val_loss: 2.1154
Epoch 3/15
625/625 ————— 2s 3ms/step - accuracy: 0.1672 - loss: 2.1793 - val_
accuracy: 0.2291 - val_loss: 2.0906
Epoch 4/15
625/625 ————— 2s 3ms/step - accuracy: 0.1761 - loss: 2.1554 - val_
accuracy: 0.2406 - val_loss: 2.0832
Epoch 5/15
625/625 ————— 2s 3ms/step - accuracy: 0.1733 - loss: 2.1691 - val_
accuracy: 0.2362 - val_loss: 2.1059
Epoch 6/15
625/625 ————— 2s 3ms/step - accuracy: 0.1788 - loss: 2.1555 - val_
accuracy: 0.2037 - val_loss: 2.1129
Epoch 7/15
625/625 ————— 2s 3ms/step - accuracy: 0.1771 - loss: 2.1643 - val_
accuracy: 0.2464 - val_loss: 2.0917
Epoch 8/15
625/625 ————— 2s 3ms/step - accuracy: 0.1782 - loss: 2.1608 - val_
accuracy: 0.2063 - val_loss: 2.1517
Epoch 9/15
625/625 ————— 2s 3ms/step - accuracy: 0.1847 - loss: 2.1506 - val_
accuracy: 0.1856 - val_loss: 2.1564
Epoch 10/15
625/625 ————— 2s 3ms/step - accuracy: 0.1812 - loss: 2.1611 - val_
accuracy: 0.1955 - val_loss: 2.1351
Epoch 11/15
625/625 ————— 2s 3ms/step - accuracy: 0.1859 - loss: 2.1523 - val_
accuracy: 0.2052 - val_loss: 2.1374
Epoch 12/15
625/625 ————— 2s 3ms/step - accuracy: 0.1792 - loss: 2.1611 - val_
accuracy: 0.2068 - val_loss: 2.1508
Epoch 13/15
625/625 ————— 2s 3ms/step - accuracy: 0.1826 - loss: 2.1481 - val_
accuracy: 0.1788 - val_loss: 2.2142
Epoch 14/15
625/625 ————— 2s 3ms/step - accuracy: 0.1873 - loss: 2.1400 - val_
accuracy: 0.1620 - val_loss: 2.2375
Epoch 15/15
625/625 ————— 2s 3ms/step - accuracy: 0.1772 - loss: 2.1653 - val_
accuracy: 0.1763 - val_loss: 2.1727
Regularized Model -> Final Train Accuracy: 0.1658 | Final Val Accuracy: 0.1763

```

4.2 Regularization Techniques (Dropout vs. L2 Regularization):

- [You can insert related screenshots here if you have them]
- **Dropout Layer:** Randomly turns off (sets to zero) a fraction of neurons during training. **Pros:** Forces the network to learn robust features and strongly prevents overfitting. **Cons:** Increases training time.
- **L2 Regularization:** Adds a weight penalty (squared magnitude of weights) to the loss function. **Pros:** Keeps all weights small, restricting model complexity.

Cons: Requires tuning the penalty parameter (λ).

- **Key Differences:** Dropout prevents overfitting by dynamically turning off neurons (changing network structure). L2 prevents overfitting by mathematically forcing weights to stay small (changing the loss equation).

5. Reflection on Training Dynamics

After running experiments and analyzing your results, reflect on the following:

Questions to Reflect On:

1. How did changing the learning rate affect the model's ability to converge?
2. What effect did dropout and L2 regularization have on the model's performance?
3. How do you decide the best combination of hyperparameters for your model?

Answer:

1. **Learning Rate:** A high learning rate causes divergence (fails to converge), while a low one slows down training. An optimal rate ensures fast and stable convergence.
2. **Dropout & L2:** They reduce overfitting by preventing neuron co-adaptation (Dropout) and penalizing large weights (L2), which improves validation accuracy and generalization.
3. **Hyperparameter Tuning:** By using systematic experimentation (e.g., Grid Search or Random Search) and selecting the combination that yields the best performance on the validation set.

Programming Assignment:

Write a Python script to:

1. Experiment with at least two different learning rates.
2. Add a dropout layer to your FCNN model.
3. Train the modified model and compare its performance to the original model.

Below is a **skeleton code snippet** for the dropout modification:

```
In [3]: # Task 1: Experiment with at least two different learning rates.
# Task 2: Add a dropout layer to your FCNN model.
# Task 3: Train the modified model and compare its performance to the original m

from tensorflow.keras.layers import Dropout, Flatten, Dense
from tensorflow.keras.regularizers import l2
```

```

from tensorflow.keras.models import Sequential
from tensorflow.keras.optimizers import Adam

# Defining a list of two Learning rates to compare their impact on model converg
learning_rates = [0.01, 0.001]

for lr in learning_rates:
    print(f"\n--- Training Modified Model with Learning Rate: {lr} ---")

    # Recreate the model inside the loop to ensure fresh weights for each experi
    model = Sequential([
        Flatten(input_shape=(32, 32, 3)),
        # Adding L2 Regularization (weight decay) as an extra measure to prevent
        Dense(128, activation='relu', kernel_regularizer=l2(0.01)),

        # This Layer randomly deactivates 50% of neurons during training to prev
        Dropout(0.5),
        Dense(64, activation='relu'),
        Dense(10, activation='softmax')
    ])

    # Compile the model with the current Learning rate from the loop
    model.compile(
        optimizer=Adam(learning_rate=lr),
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )

    # Training the model and saving the history object to extract final accuraci
    history = model.fit(
        train_images,
        train_labels,
        validation_data=(val_images, val_labels),
        epochs=20,
        batch_size=64,
        verbose=0 # Hiding the progress bar to keep output clean, showing final
    )

    # Extract final training and validation accuracy to evaluate and compare per
    final_train_acc = history.history['accuracy'][-1]
    final_val_acc = history.history['val_accuracy'][-1]

    print(f"Result for LR {lr}:")
    print(f"  -> Final Train Accuracy: {final_train_acc:.4f}")
    print(f"  -> Final Val Accuracy:    {final_val_acc:.4f}")

```

```
--- Training Modified Model with Learning Rate: 0.01 ---
```

```
Result for LR 0.01:
```

```
-> Final Train Accuracy: 0.1012
```

```
-> Final Val Accuracy:    0.0996
```

```
--- Training Modified Model with Learning Rate: 0.001 ---
```

```
Result for LR 0.001:
```

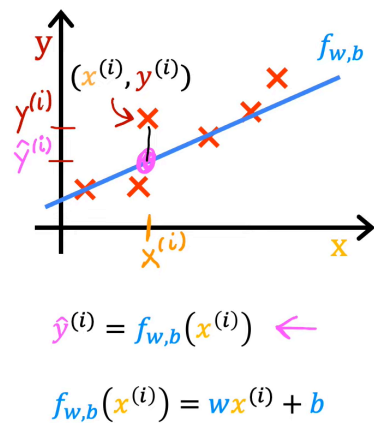
```
-> Final Train Accuracy: 0.1921
```

```
-> Final Val Accuracy:    0.1482
```

Question 1: Mathematical Formulas of Loss Functions

Mean Squared Error (MSE):

- Formula: $MSE = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$



Cost function: Squared error cost function

$$J(w,b) = \frac{1}{2m} \sum_{i=1}^m \left(\hat{y}^{(i)} - y^{(i)} \right)^2$$

m = number of training examples

$$J(w,b) = \frac{1}{2m} \sum_{i=1}^m \left(f_{w,b}(x^{(i)}) - y^{(i)} \right)^2$$

↑ intuition

Find w, b :

$\hat{y}^{(i)}$ is close to $y^{(i)}$ for all $(x^{(i)}, y^{(i)})$.

Stanford ONLINE

DeepLearning.AI

Andrew Ng

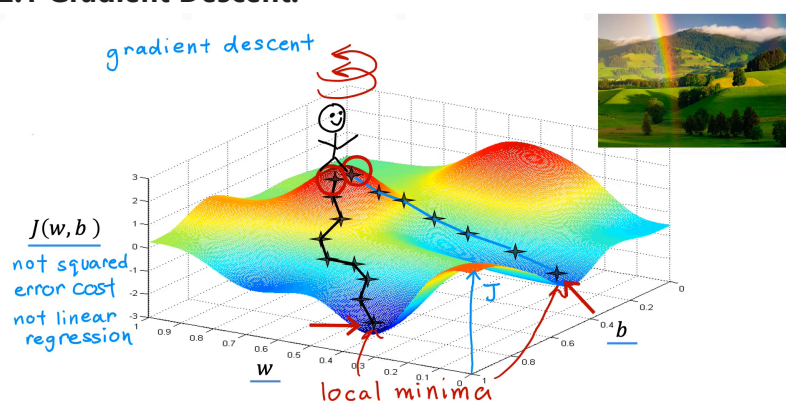
- Explanation: It measures the average squared difference between the predicted values ($\hat{y}$) and the true values (y). It is best suited for **continuous regression tasks** (e.g., predicting prices) because it penalizes large errors.

Cross-Entropy Loss:

- Formula: $CE = - \sum_{i=1}^C y_i \log(\hat{y}_i)$
- Explanation: It measures the difference between two probability distributions. It heavily penalizes models that are "confidently wrong" by using the $-\log$ function. It is best suited for **classification tasks**.

Question 2: Andrew Ng Course Slides & Explanations

2.1 Gradient Descent:



Stanford ONLINE

DeepLearning.AI

Andrew Ng

Have some function $J(w, b)$ *for linear regression or any function*
 Want $\min_{w, b} J(w, b)$ $\min_{w_1, \dots, w_n, b} J(w_1, w_2, \dots, w_n, b)$

Outline:

Start with some w, b (set $w=0, b=0$)
 Keep changing w, b to reduce $J(w, b)$ *J not always*
 Until we settle at or near a minimum *may have >1 minimum*

Stanford ONLINE DeepLearning.AI

Andrew Ng

- **Explanation:** Gradient descent is an optimization algorithm used to minimize the loss function. It calculates the gradient (slope) of the loss and takes small steps in the opposite direction to find the lowest possible error.

2.2 Binary Cross-Entropy Loss:

Cost

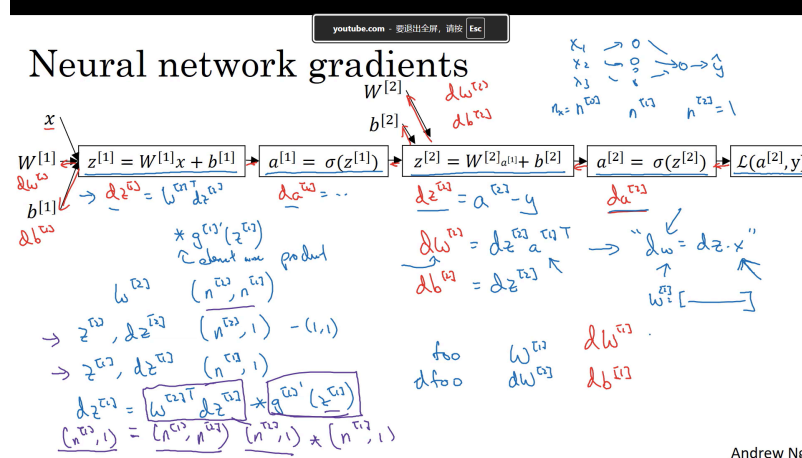
$$J(\vec{w}, b) = \frac{1}{m} \sum_{i=1}^m L(\underbrace{f_{\vec{w}, b}(\vec{x}^{(i)})}_{\text{loss}}, y^{(i)})$$

$$= \begin{cases} -\log(f_{\vec{w}, b}(\vec{x}^{(i)})) & \text{if } y^{(i)} = 1 \text{ Convex} \\ -\log(1 - f_{\vec{w}, b}(\vec{x}^{(i)})) & \text{if } y^{(i)} = 0 \end{cases}$$

find w, b that minimize cost J

- **Explanation:** The math equation relies on $-\log(p)$ if the true label is 1, and $-\log(1 - p)$ if the true label is 0. If the true label is 1 but the model predicts a probability near 0, $-\log(0)$ goes to infinity, resulting in a massive penalty.

2.3 Backpropagation:



Andrew Ng

Summary of gradient descent

$\underline{dz^{[2]}} = \underline{a^{[2]}} - \underline{y}$	$\underline{dZ^{[2]}} = A^{[2]} - Y$	$J(\cdot) = \frac{1}{m} \sum_{i=1}^m J(\hat{y}_i, y_i)$
$dW^{[2]} = dz^{[2]} a^{[1]T}$	$dW^{[2]} = \frac{1}{m} dZ^{[2]} A^{[1]T}$	
$db^{[2]} = dz^{[2]}$	$db^{[2]} = \frac{1}{m} np.sum(dZ^{[2]}, axis = 1, keepdims = True)$	
$\underset{(n^{[1]}, 1)}{dz^{[1]}} = W^{[2]T} dz^{[2]} * \underset{(n^{[1]}, m)}{g^{[1]}'(Z^{[1]})}$	$\underset{(n^{[1]}, m)}{dZ^{[1]}} = \underset{(n^{[2]}, m)}{W^{[2]T} dz^{[2]}} * \underset{(n^{[1]}, m)}{g^{[1]}'(Z^{[1]})}$	$\downarrow$ element-wise product
$dW^{[1]} = dz^{[1]} X^T$	$dW^{[1]} = \frac{1}{m} dZ^{[1]} X^T$	
$db^{[1]} = dz^{[1]}$	$db^{[1]} = \frac{1}{m} np.sum(dZ^{[1]}, axis = 1, keepdims = True)$	

- **Explanation:** Backpropagation calculates the gradient of the loss function with respect to each weight. It uses the "chain rule" from calculus to pass the error *backwards* from the output layer to the input layer, telling the model how to adjust weights to reduce the loss.